Filter type	How to short-circuit?	What else runs?
Authorization filters	Set `context.Result`.	Runs only `IAlwaysRunResultFilters`.
Resource filters	Set `context.Result`.	Resource-filter `*Executed` functions from earlier filters run with `context.Cancelled = true`. Runs `IAlwaysRunResultFilters` before executing the `IActionResult`.
Action filters	Set `context.Result`.	Bypasses only action method execution. Action filters earlier in the pipeline run their `*Executed` methods with `context.Cancelled = true`, then result filters, result execution, and resource filters' `*Executed` methods all run as normal.
Page filters	Set `context.Result` in `OnPageHandlerSelected`.	Bypasses only page handler execution. Page filters earlier in the pipeline run their `*Executed` methods with `context.Cancelled = true`, then result filters, result execution, and resource filters' `*Executed` methods all run as normal.
Exception filters	Set `context.Result` and `Exception.Handled = true`.	All resource-filter `*Executed` functions run. Runs `IAlwaysRunResultFilters` before executing the `IActionResult`.
Result filters	Set `context.Cancelled = true`.	Result filters earlier in the pipeline run their `*Executed` functions with `context.Cancelled = true`. All resource-filter `*Executed` functions run as normal.

The most interesting point here is that short-circuiting an action filter (or a page filter) doesn't short-circuit much of the pipeline at all. In fact, it bypasses only later action filters and the action method execution itself. By building primarily action filters, you can ensure that other filters, such as result filters that define the output format, run as usual, even when your action filters short-circuit.